`XYPOL(x, y) : length, angle32bit`	Convert (x,y) to (length,angle32bit)
`QSIN(length, angle, twopi) : y`	Rotate (length,0) by (angle / twopi) * 2Pi and return y. Use 0 for twopi = $1_0000_0000. Twopi is unsigned.
`QCOS(length, angle, twopi) : x`	Rotate (length,0) by (angle / twopi) * 2Pi and return x. Use 0 for twopi = $1_0000_0000. Twopi is unsigned.
`MULDIV64(mult1,mult2,divisor) : quotient`	Divide the 64-bit product of 'mult1' and 'mult2' by 'divisor', return quotient (unsigned operation)
`GETRND() : Rnd`	Get random long (from xoroshiro128** PRNG, seeded on boot with thermal noise from ADC)

Memory Methods	Details
`GETREGS(HubAddr, CogAddr, Count)`	Move Count registers at CogAddr to longs at HubAddr
`SETREGS(HubAddr, CogAddr, Count)`	Move Count longs at HubAddr to registers at CogAddr
`BYTEMOVE(Dest, Source, Count)`	Move Count bytes from Source to Dest
`WORDMOVE(Dest, Source, Count)`	Move Count words from Source to Dest
`LONGMOVE(Dest, Source, Count)`	Move Count longs from Source to Dest
`BYTEFILL(Dest, Value, Count)`	Fill Count bytes at Dest with Value
`WORDFILL(Dest, Value, Count)`	Fill Count words at Dest with Value
`LONGFILL(Dest, Value, Count)`	Fill Count longs at Dest with Value

String Methods	Details
`STRSIZE(Addr) : Size`	Count bytes in zero-terminated string at Addr, return string size, not including zero terminator
`STRCOMP(AddrA,AddrB) : Match`	Compare zero-terminated strings at AddrA and AddrB, return -1 if match or 0 if mismatch
`STRING("Text",9) : StringAddress`	Compose a zero-terminated string (quoted characters and values 1..255 allowed), return address of string

Index ↔ Value Methods	Details
`LOOKUP(Index: v1, v2..v3, etc) : Value`	Lookup value (values and ranges allowed) using 1-based index, return value (0 if index out of range)
`LOOKUPZ(Index: v1, v2..v3, etc) : Value`	Lookup value (values and ranges allowed) using 0-based index, return value (0 if index out of range)
`LOOKDOWN(Value: v1, v2..v3, etc) : Index`	Determine 1-based index of matching value (values and ranges allowed), return index (0 if no match)
`LOOKDOWNZ(Value: v1, v2..v3, etc) : Index`	Determine 0-based index of matching value (values and ranges allowed), return index (0 if no match)

USING METHODS

Methods that return single results can be used as terms in expressions:

```
x := GETRND() +// 100    'Get a random number between 0 and 99

BYTEMOVE(ToStr, FromStr, STRSIZE(FromStr) + 1)
```

Methods which return multiple results (like POLXY) can be used to supply multiple parameters to other methods:

```
x,y := SumPoints(POLXY(rho1,theta1), POLXY(rho2,theta2))
```

…**where**…

```
SumPoints(x1, y1, x2, y2) : x, y
  RETURN x1+x2, y1+y2
```

Multiple method results can be assigned to variables or ignored by using an underscore in lieu of a variable name::

```
x,y := ROTXY(xin,yin,theta)     'use both the x and y results
_,y := ROTXY(xin,yin,theta)     'use only the y result
x,_ := ROTXY(xin,yin,theta)     'use only the x result
```

User-defined methods which return one or more results can also be used as instructions, where the return values are ignored. However, built-in methods such as STRSIZE, which return results, must be used as expression terms.

ABORT

Spin2 has an "abort" mechanism for instantly returning, from any depth of nested method calls, back to a base caller which used '\' before the method name. A single return value can be conveyed from the abort point back to the base caller:

```
PRI Sub1() : Error    'Sub1 calls Sub2 with an ABORT trap
  Error := \Sub2()    '\ means call method and trap any ABORT
  \Sub2()             'in this case, the ABORT value is ignored

PRI Sub2()            'Sub2 calls Sub3
  Sub3()              'Sub3 never returns here due to the ABORT
  PINHIGH(0)          'PINHIGH never executes

PRI Sub3()            'Sub3 ABORTs, returning to Sub1 with ErrorCode
  ABORT ErrorCode     'ABORT and return ErrorCode
  PINLOW(0)           'PINLOW never executes
```

Regardless of how many return values a particular method may have, when that method is called with a preceding "\", there will be only one return value, which may be ignored.

If no value is specified after ABORT, then zero will be returned.

If a method is called with a preceding "\", but no ABORT occurs, then zero will be returned.

If an ABORT executes without a "\" trap somewhere in the call chain, the cog returns past the top-level method and executes COGSTOP(COGID), shutting itself down.

The abort mechanism is intended as a means to return from a deeply nested subroutine where some error situation has developed, but it can be used for any purpose. Basically, it's a way to return to a base caller without having to check for a condition to do so at every level of the call chain. It returns all the way back to the caller with the "\" abort trap, carrying the ABORT value. You can compose hierarchical levels of "\" abort traps and ABORT points.

METHOD POINTERS

Method pointers are LONG values which point to a method and are then used to call that method indirectly.

To establish a method pointer, you can assign a long variable using "@" before the method name. Note that there are no parentheses after the method name:

```
LongVar := @SomeMethod                          'a method within the current object
LongVar := @SomeObject.SomeMethod               'a method within a child object
LongVar := @SomeOject[index].SomeMethod         'a method within an indexed child object
```

Method pointers can be generated on-the-fly and passed as parameters:

```
SetUpIO(@InMethod,@OutMethod)
```

Method pointers are then used in the following ways to call methods:

```
LongVar()                                       'no parameters and no return values
LongVar(Par1, Par2)                             'two parameters and no return values
Var := LongVar():1                              'no parameters and one return value
Var1,Var2 := LongVar(Par1):2                    'one parameters and two return values
Var1,Var2 := POLXY(LongVar(Par1,Par2,Par3):2)   'three parameters and two return values
```

There is no compile-time awareness of how many parameters the method pointed to actually has. You need to code your method pointer usage such that you supply the proper number of parameters and specify the proper number of return values after a ":", so that there is agreement with the method pointed to.

Method pointers can be passed through object hierarchies to enable direct calling of any method from anywhere. They can also be used to dynamically point to different methods which have the same numbers of parameters and return values.

How Method Pointers Work

An @method expression generates a 32-bit value which has two bit fields:

[31..20] = Index of the method, relative to the method's object base. The index of the first method will be twice the number of objects instantiated

[19..0] = Address of the method's VAR base. The method's VAR base, in turn, contains the address of the method's object base.

By putting the method's index and VAR base address together into the 32-bit value, and having the VAR base contain the method's object base address, a complete method pointer is established in a single long, which can be treated as any other variable.

To accommodate method pointers, each object instance reserves the first long of its VAR space for the object base address. When an @method expression executes, that first long is written with the object's base address.

SEND

SEND is a special method pointer which is inherited from the calling method and, in turn, conveyed to all called methods. It's purpose is to provide an efficient output mechanism for data.

SEND can be assigned like a method pointer, but it must point to a method which takes one parameter and has no return values:

```
SEND := @OutMethod
```

When used as a method, SEND will pass all parameters, including any return values from called methods, to the method SEND points to:

```
SEND("Hello! ", GetDigit()+"0", 13)
```

Any methods called within the SEND parameters will inherit the SEND pointer, so that they can do SEND methods, too:

```
PUB Go()
  SEND := @SetLED
  REPEAT
    SEND(Flash(),$01,$02,$04,$08,$10,$20,$40,$80)

PRI Flash() : x
  REPEAT 2
    SEND($00,$FF,$00)
  RETURN $AA

PRI SetLED(x)
  PINWRITE(56 ADDPINS 7, !x)
  WAITMS(125)
```

In the above example, the following values are output in repeating sequence: $00, $FF, $00, $00, $FF, $00, $AA, $01, $02, $04, $08, $10, $20, $40, $80 (but inverted for LEDs)

Though a called method inherits the current SEND pointer, it may change it for its own purposes. Upon return from that method, the SEND pointer will be back to what it was before the method was called. So, the SEND pointer value is propagated in method calls, but not in method returns.

RECV

RECV, like SEND, is a special method pointer which is inherited from the calling method and, in turn, conveyed to all called methods. It's purpose is to provide an efficient input mechanism for data.

RECV can be assigned like a method pointer, but it must point to a method which takes no parameters and returns a single value:

```
RECV := @InMethod
```

An example of using RECV:

```
VAR i

PUB Go()
  RECV := @GetPattern
  REPEAT
    PINWRITE(56 ADDPINS 7, !RECV())
    WAITMS(125)

PRI GetPattern() : Pattern
  RETURN DECOD(i++ & 7)
```

In the above example, the following values are output in repeating sequence: $01, $02, $04, $08, $10, $20, $40, $80 (but inverted for LEDs)

Though a called method inherits the current RECV pointer, it may change it for its own purposes. Upon return from that method, the RECV pointer will be back to what it was before the method was called. So, the RECV pointer value is propagated in method calls, but not in method returns.

FLOW CONTROL

Spin2 has three basic flow-control constructs:

IF / IFNOT + ELSEIF / ELSEIFNOT + ELSE	- Conditional execution with random decision logic
CASE / CASE_FAST	- Conditional execution with single target and multiple match tests
REPEAT	- Looped execution with various modes

All these constructs use relative indentation to determine which code falls under their control:

```
IF cog                                'if cog <> 0
  COGSTOP(cog-1)                      '..then stop cog
  PINCLEAR(av_base_pin_ ADDPINS 4)    '..then clear pin mode(s)
```

The flow-control constructs can be nested in any order:

```
CASE flag
  0: CASE_FAST chr
       0:     BYTEFILL(@screen, " ", screen_size)
              col := row := 0
       1:     col := row := 0
       2..7:  flag := chr
              RETURN
       8:     IF col
                col--
       9:     REPEAT
                out(" ")
              WHILE col & 7
       10:    RETURN
       11:    color := $00
       12:    color := $80
       13:    newline()
       OTHER: out(chr)

    2:    col := chr // cols
    3:    row := chr // rows
    4..7: background0_[flag-$04] := chr << 8
  flag := 0
```

IF / IFNOT + ELSEIF / ELSEIFNOT + ELSE

The IF construct begins with IF or IFNOT and optionally employs ELSEIF, ELSEIFNOT, and ELSE. To all be part of the same decision tree, these keywords must have the same level of indentation.

The indented code under IF or ELSEIF executes if <condition> is not zero. The code under IFNOT or ELSEIFNOT executes if <condition> is zero. The code under ELSE executes if no other indented code executed:

IF / IFNOT <condition> **<indented code>**	- Initial IF or IFNOT
ELSEIF / ELSEIFNOT <condition> **<indented code>**	- Optional ELSEIF or ELSEIFNOT
ELSE	- Optional final ELSE

```
  <indented code>
```

CASE / CASE_FAST

The CASE construct sequentially compares a target value to a list of possible matches. When a match is found, the related code executes.

Match values/ranges must be indented past the CASE keyword. Multiple match values/ranges can be expressed with comma separators. Any additional lines of code related to the match value/range must be indented past the match value/range:

```
CASE target                             - CASE with target value
  <match> : <code>                      - match value and code
    <indented code>
  <match..match> : <code>               - match range and code
    <indented code>
  <match>,<match..match> : <code>       - match value, range, and code
    <indented code>
  OTHER : <code>                        - optional OTHER case, in case no match found
    <indented code>
```

CASE_FAST is like CASE, but rather than sequentially comparing the target to a list of possible matches, it uses an indexed jump table of up to 256 entries to immediately branch to the appropriate code, saving time at a possible cost of larger compiled code. If there are only contiguous match values and no match ranges, the resulting code will actually be smaller than a normal CASE construct with more than several match values.

For CASE_FAST to compile, the match values/ranges must be unique constants which are all within 255 of each other.

See CASE_FAST example under "FLOW CONTROL" above.

REPEAT

All looping is achieved through REPEAT constructs, which have several forms:

```
REPEAT                                  - Repeat forever (useful for putting at end of program if you don't want the cog to stop and cease driving its I/O's)
  <indented code>

REPEAT <count>                          - Repeat <count> times, if <count> is zero then <indented code> is skipped
  <indented code>

REPEAT <variable> FROM <first> TO <last>                        - Repeat while iterating <variable> from <first> to <last>, stepping by +/-1
  <indented code>                                               - After completion, <variable> = <last> +/-1

REPEAT <variable> FROM <first> TO <last> STEP <delta>           - Repeat while iterating <variable> from <first> to <last>, stepping by +/-<delta>
  <indented code>                                               - After completion, <variable> = <last> +/-<delta>

REPEAT WHILE <condition>                - Repeat while <condition> is not zero, <condition> is evaluated before <indented code> executes
  <indented code>

REPEAT UNTIL <condition>                - Repeat until <condition> is not zero, <condition> is evaluated before <indented code> executes
  <indented code>

REPEAT                                  - Repeat while <condition> is not zero, <condition> is evaluated after <indented code> executes
  <indented code>
WHILE <condition>                       - WHILE must have same indentation as REPEAT

REPEAT                                  - Repeat until <condition> is not zero, <condition> is evaluated after <indented code> executes
  <indented code>
UNTIL <condition>                       - UNTIL must have same indentation as REPEAT
```

Within REPEAT constructs, there are two special instructions which can be used to change the course of execution: NEXT and QUIT. NEXT will immediately branch to the point in the REPEAT construct where the decision to loop again is made, while QUIT will exit the REPEAT construct and continue after it. These instructions are usually used conditionally:

```
REPEAT
  <indented code>
  IF <condition>                        - Optionally force the next iteration of the REPEAT
    NEXT
```

```
<indented code>
IF <condition>                          - Optionally quit the REPEAT
  QUIT
<indented code>
```

IN-LINE PASM CODE

Spin2 methods can execute in-line PASM code by preceding the PASM code with an **'ORG** {$000..$12F}' and terminating it with an **END**.

```
PUB go() | x

  REPEAT

    ORG
        GETRND  WC      'rotate a random bit into x
        RCL     x,#1
    END

    PINWRITE(56 ADDPINS 7, x)     'output x to the P2 Eval board's LEDs
    WAITMS(100)
```

Your PASM code will be assembled with a RET instruction added at the end to ensure that it returns to Spin2, in case no early _RET_ or RET executes.

Here's the internal Spin2 procedure for executing in-line PASM code:

- Save the current streamer address for restoration after the PASM code executes.
- Copy the method's first 16 long variables, including any parameters, return values, and local variables, from hub RAM to cog registers $1E0..$1EF.
- Copy the in-line PASM-code longs from hub RAM into cog registers, starting at the ORG address (default is $000).
- CALL the PASM code.
- Restore the 16 longs in cog registers $1E0..$1EF back to hub RAM, in order to update any modified method variables.
- Restore the streamer address and resume Spin2 bytecode execution.

Within your in-line PASM code, you can do all these things:

- Read and write the following register areas:
    - $000..$12F, which your PASM code loads into. You can even load different PASM programs at different addresses within this range and CALL them from Spin2.
    - $1D8..$1DF, which are general-purpose registers, named PR0..PR7, available to both PASM and Spin2 code.
    - $1E0..$1EF, which temporarily contain the method's first 16 long hub RAM variables and are temporarily assigned the same symbolic names.
    - $1F0..$1FF, which include IJMP3, IRET3, IJMP2, IRET2, IJMP1, IRET1, PA, PB, PTRA, PTRB, DIRA, DIRB, OUTA, OUTB, INA, and INB.
    - Avoid writing to $130..$1D7 and LUT RAM, since the Spin2 interpreter occupies these areas. You can look in "Spin2_interpreter.spin2" to see the interpreter code.
- Use the streamer temporarily.
- Use up to 5 levels of the hardware stack for nested CALLs, including CALLs to hub RAM.
- Declare and reference regular and local symbols. These symbols will not be accessible outside of your PASM code.
- Declare BYTE, WORD, and LONG data.
- Use the RES, ORGF, and FIT directives. The directives ORG, ORGH, ALIGNW, ALIGNL, and FILE are not allowed within in-line PASM code.
- Establish an interrupt which executes your code remaining in cog registers $000..$12F. Spin2 accommodates interrupts and only stalls them briefly, when necessary.
- Return to Spin2, at any point, by executing an _RET_ or RET instruction.

CALLING PASM FROM SPIN2

You can do a **CALL(address)** in Spin2 to execute PASM code in either cog register space or hub RAM.

```
PUB go() | x

  REPEAT
    CALL(@random)
    PINWRITE(56 ADDPINS 7, PR0)
    WAITMS(100)


DAT     ORGH    'hub PASM program to rotate a random bit into pr0

random  GETRND  WC
 _RET_  RCL     PR0,#1
```

Here's the internal Spin2 procedure for executing a CALL:

- Save the current streamer address for restoration after the PASM code executes.
- CALL the PASM code.
- Restore the streamer address and resume Spin2 bytecode execution.

Within code which you CALL, you can do all these things:

- Read and write the following register areas:
    - $000..$12F, which may contain PASM code and/or data which you previously loaded.
    - $1D8..$1DF, which are general-purpose registers, named PR0..PR7, available to both PASM and Spin2 code.
    - $1E0..$1EF, which are available for scratchpad use, but will likely be rewritten when Spin2 resumes.
    - $1F0..$1FF, which include IJMP3, IRET3, IJMP2, IRET2, IJMP1, IRET1, PA, PB, PTRA, PTRB, DIRA, DIRB, OUTA, OUTB, INA, and INB.
    - Avoid writing to $130..$1D7 and LUT RAM, since the Spin2 interpreter occupies these areas. You can look in "Spin2_interpreter.spin2" to see the interpreter code.
- Use the streamer temporarily.
- Use up to 5 levels of the hardware stack for nested CALLs, including CALLs to hub RAM.
- Establish an interrupt which executes your code remaining in cog registers $000..$12F. Spin2 accommodates interrupts and only stalls them briefly, when necessary.
- Return to Spin2, at any point, by executing an _RET_ or RET instruction.

REGLOAD and REGEXEC

The Spin2 instructions **REGLOAD(HubAddress)** and **REGEXEC(HubAddress)** are used to load or load-and-execute PASM code and/or data chunks from hub RAM into cog registers.

The chunk of PASM code and/or data must be preceded with two words which provide the starting register and the number of registers (longs) to load, minus 1.

```
PUB go()

  REGLOAD(@chunk)    'load self-defined chunk from hub into registers

  REPEAT
    CALL(#start)     'call program within chunk at register address
    WAITMS(100)

DAT

chunk   WORD    start,finish-start-1  'define chunk start and size-1
        ORG     $120                  'org can be $000..$130-size

start   DRVRND  #56 ADDPINS 7         'some code
 _RET_  DRVNOT  #0                    'more code + return
finish
```

REGEXEC works like REGLOAD, but it also CALLs to the start register of the chunk after loading it.

In the example below, REGEXEC launches a chunk of code in upper register memory which sets up a timer interrupt and then returns to Spin2. Meanwhile, as the Spin2 method repeatedly randomizes pins 60..63 every 100ms, the chunk of code loaded into upper register memory perpetuates the timer interrupt and toggles pins 56..59 every 500ms. Note that registers $000..$127 are still free for other code chunks and interrupts 2 and 3 are still unused.

```
PUB go()

  REGEXEC(@chunk)                      'load self-defined chunk and execute it
                                       'chunk starts timer interrupt and returns
  REPEAT
    PINWRITE(60 ADDPINS 3, GETRND())   'randomize pins 60..63
    WAITMS(100)                        'pins 56..59 toggle via interrupt

DAT

chunk   WORD    start,finish-start-1   'define chunk start and size-1
        ORG     $128                   'org can be $000..$130-size

start   MOV     IJMP1,#isr             'set int1 vector
        SETINT1 #1                     'set int1 to ct-passed-ct1 event
        GETCT   PR0                    'get ct
 _ret_  ADDCT1  PR0,bigwait            'set initial ct1 target, return to Spin2

isr     DRVNOT  #56 ADDPINS 3          'interrupt service routine, toggle 56..59
        ADDCT1  PR0,bigwait            'set next ct1 target
        RETI1                          'return from interrupt

bigwait LONG    20_000_000 / 2         '500ms second on RCFAST
finish
```

DEBUG

The Spin2 compiler contains a stealthy debugger program that can be automatically downloaded with your application. It uses the last 16KB of RAM plus a few bytes for each Spin2 DEBUG statement and one instruction for each PASM DEBUG statement. You place DEBUG statements in your application which contain output commands that will serially transmit the state of variables and equations as your application runs. Each time a DEBUG statement is encountered during execution, the debugger is invoked and it outputs the message for that statement. Debugging is initiated in PNut by adding the Ctrl key to the usual F10 to 'run' or F11 to 'program', or in Propeller Tool by enabling Debug Mode with Ctrl+D then using F10 or F11 as is normal. This compiles your application with all the DEBUG statements, adds the debugger to the download, and then brings up the DEBUG Output window which begins receiving messages at the start of your application.

Things to know about the DEBUG system

- To use the debugger, you must configure at least a 10 MHz clock derived from a crystal or external input. You cannot use RCFAST or RCSLOW.
- The debugger occupies the top 16 KB of hub RAM, remapped to $FC000..$FFFFF and write-protected. The hub RAM at $7C000..$7FFFF will no longer be available.
- Data defining each DEBUG statement is stored within the debugger image in the top 16 KB of RAM, minimizing impact on your application code.
- In Spin2, each DEBUG statement adds three bytes, plus any code needed to reference variables and resolve run-time expressions used in the DEBUG statement.
- In PASM, each DEBUG statement adds one instruction (long).
- DEBUG statements are ignored by the compiler when not compiling for DEBUG mode, so you don't need to comment them out when debugging is not in use.
- If no DEBUG statements exist in your application, you will still get notification messages when cogs are started.
- Debugging is invoked by holding CTRL (in PNut), or enabling Debug with CTRL+D (in Propeller Tool), before the usual F9..F11 keys, tocompile, download, and program to flash.
- During execution, as DEBUG statements are encountered, text messages are sent out serially on P62 at 2 Mbaud in 8-N-1 format.
- DEBUG messages always start with "CogN ", where N is the cog number, followed by two spaces, and they always end with CR+LF (new line).
- Up to 255 DEBUG statements can exist within your application, since the BRK instruction is used to interrupt and select the particular DEBUG statement definition.
- You can define several symbols to modify debugger behavior: DEBUG_COG, DEBUG_DELAY, DEBUG_BAUD, DEBUG_PIN, DEBUG_TIMESTAMP, etc. See table.
- Each time a debug-enabled cog is started, a debug message is output to indicate the cog number, code address (PTRB), parameter (PTRA), and 'load' or 'jump' mode.
- For Spin2, DEBUG statements can output expression and variable values, hub byte/word/long arrays, and register arrays.
- For PASM, DEBUG statements can output register values/arrays, hub byte/word/long arrays, and constants. PASM syntax is used: implied register or #immediate.
- DEBUG output data can be displayed in decimal, hex, or binary, signed or unsigned, and sized to byte, word, long, or auto. Hub character strings are also supported.
- DEBUG output commands show both the source and value: "DEBUG(UHEX(x))" might output "x = $123".
- DEBUG commands which output data can have multiple sets of parameters, separated by commas: SDEC(x,y,z) and LSTR(ptr1,size1,ptr2,size2)
- Commas are automatically output between data: "DEBUG(UHEX_BYTE(d,e,f), SDEC(g))" might output "d = $45, e = $67, f = $89, g = -1_024".
- All DEBUG output commands have alternate versions, ending in "_" which output only the value: DEBUG(UHEX_BYTE_(d,e,f)) might output "$45, $67, $89".
- DEBUG statements can contain comma-separated strings and characters, aside from commands: DEBUG(`"We got here! Oh, Nooooo...", 13, 13`)
- DEBUG statements may contain IF() and IFNOT() commands to gate further output within the statement. An initial IF/IFNOT will gate the entire message.
- DEBUG statements may contain a final DLY(milliseconds) command to slow down a cog's messaging, since messages may stream at the rate of ~10,000 per second.
- DEBUG serial output can be redirected to a different pin, at a different baud rate, for displaying/logging elsewhere.
- LOCK[15] is allocated by the debugger and used among all cogs during their debug interrupts to time-share the DEBUG serial-transmit pin.
- Command-line supports DEBUG-only mode: PNut -debug {CommPort if not 1} {BaudRate if not 2_000_000}

Commands for use in DEBUG statements

Conditionals	Details
`IF(condition)`	If condition <> 0 then continue at the next command within the DEBUG statement, else skip all remaining commands and output CR+LF. If used as the first command in the DEBUG statement, IF will gate ALL output for the statement, including the "CogN "+CR+LF. This way, DEBUG messages can be entirely suppressed, so that you can filter what is important.
`IFNOT(condition)`	If condition = 0 then continue at the next command within the DEBUG statement, else skip all remaining commands and output CR+LF. If used as the first command in the DEBUG statement, IFNOT will gate ALL output for the statement, including the "CogN "+CR+LF. This way, DEBUG messages can be entirely suppressed, so that you can filter what is important.

String Output *	Details	Output
`ZSTR(hub_pointer)`	Output zero-terminated string at hub_pointer	`"Hello!"`
`LSTR(hub_pointer,size)`	Output 'size' characters of string at hub_pointer	`"Goodbye."`

Decimal Output, unsigned *	Details	Min Output	Max Output
`UDEC(value)`	Output unsigned decimal value	`0`	`4_294_967_295`
`UDEC_BYTE(value)`	Output byte-size unsigned decimal value	`0`	`255`
`UDEC_WORD(value)`	Output word-size unsigned decimal value	`0`	`65_535`
`UDEC_LONG(value)`	Output long-size unsigned decimal value	`0`	`4_294_967_295`
`UDEC_REG_ARRAY(reg_pointer,size)`	Output register array as unsigned decimal values	`0`	`4_294_967_295`
`UDEC_BYTE_ARRAY(hub_pointer,size)`	Output hub byte array as unsigned decimal values	`0`	`255`
`UDEC_WORD_ARRAY(hub_pointer,size)`	Output hub word array as unsigned decimal values	`0`	`65_535`
`UDEC_LONG_ARRAY(hub_pointer,size)`	Output hub long array as unsigned decimal values	`0`	`4_294_967_295`
Decimal Output, signed *	**Details**	**Min Output**	**Max Output**

SDEC(value)	Output signed decimal value	-2_147_483_648	2_147_483_647
SDEC_BYTE(value)	Output byte-size signed decimal value	-128	127
SDEC_WORD(value)	Output word-size signed decimal value	-32_768	32_767
SDEC_LONG(value)	Output long-size signed decimal value	-2_147_483_648	2_147_483_647
SDEC_REG_ARRAY(reg_pointer,size)	Output register array as signed decimal values	-2_147_483_648	2_147_483_647
SDEC_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as signed decimal values	-128	127
SDEC_WORD_ARRAY(hub_pointer,size)	Output hub word array as signed decimal values	-32_768	32_767
SDEC_LONG_ARRAY(hub_pointer,size)	Output hub long array as signed decimal values	-2_147_483_648	2_147_483_647
Hexadecimal Output, unsigned *	**Details**	**Min Output**	**Max Output**
UHEX(value)	Output auto-size unsigned hex value	$0	$FFFF_FFFF
UHEX_BYTE(value)	Output byte-size unsigned hex value	$00	$FF
UHEX_WORD(value)	Output word-size unsigned hex value	$0000	$FFFF
UHEX_LONG(value)	Output long-size unsigned hex value	$0000_0000	$FFFF_FFFF
UHEX_REG_ARRAY(reg_pointer,size)	Output register array as unsigned hex values	$0000_0000	$FFFF_FFFF
UHEX_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as unsigned hex values	$00	$FF
UHEX_WORD_ARRAY(hub_pointer,size)	Output hub word array as unsigned hex values	$0000	$FFFF
UHEX_LONG_ARRAY(hub_pointer,size)	Output hub long array as unsigned hex values	$0000_0000	$FFFF_FFFF
Hexadecimal Output, signed *	**Details**	**Min Output**	**Max Output**
SHEX(value)	Output auto-size signed hex value	-$8000_0000	$7FFF_FFFF
SHEX_BYTE(value)	Output byte-size signed hex value	-$80	$7F
SHEX_WORD(value)	Output word-size signed hex value	-$8000	$7FFF
SHEX_LONG(value)	Output long-size signed hex value	-$8000_0000	$7FFF_FFFF
SHEX_REG_ARRAY(reg_pointer,size)	Output register array as signed hex values	-$8000_0000	$7FFF_FFFF
SHEX_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as signed hex values	-$80	$7F
SHEX_WORD_ARRAY(hub_pointer,size)	Output hub word array as signed hex values	-$8000	$7FFF
SHEX_LONG_ARRAY(hub_pointer,size)	Output hub long array as signed hex values	-$8000_0000	$7FFF_FFFF
Binary Output, unsigned *	**Details**	**Min Output**	**Max Output**
UBIN(value)	Output auto-size unsigned binary value	%0	%11111111_11111111_11111111_11111111
UBIN_BYTE(value)	Output byte-size unsigned binary value	%00000000	%11111111
UBIN_WORD(value)	Output word-size unsigned binary value	%00000000_00000000	%11111111_11111111
UBIN_LONG(value)	Output long-size unsigned binary value	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
UBIN_REG_ARRAY(reg_pointer,size)	Output register array as unsigned binary values	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
UBIN_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as unsigned binary values	%00000000	%11111111
UBIN_WORD_ARRAY(hub_pointer,size)	Output hub word array as unsigned binary values	%00000000_00000000	%11111111_11111111
UBIN_LONG_ARRAY(hub_pointer,size)	Output hub long array as unsigned binary values	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
Binary Output, signed *	**Details**	**Min Output**	**Max Output**
SBIN(value)	Output auto-size signed binary value	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_BYTE(value)	Output byte-size signed binary value	-%10000000	%01111111
SBIN_WORD(value)	Output word-size signed binary value	-%10000000_00000000	%01111111_11111111
SBIN_LONG(value)	Output long-size signed binary value	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_REG_ARRAY(reg_pointer,size)	Output register array as signed binary values	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as signed binary values	-%10000000	%01111111
SBIN_WORD_ARRAY(hub_pointer,size)	Output hub word array as signed binary values	-%10000000_00000000	%01111111_11111111
SBIN_LONG_ARRAY(hub_pointer,size)	Output hub long array as signed binary values	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111

Delay to Pace Messages	Details
DLY(milliseconds)	Delay for some milliseconds to slow down continuous message outputs for this cog. DLY is only allowed as the last command in a DEBUG statement, since it releases LOCK[15] before the delay, permitting other cogs to capture LOCK[15] so that they may take control of the DEBUG serial-transmit pin and output their own DEBUG messages.

* These commands accept multiple parameters, or multiple sets of parameters. Alternate commands with the same names, but ending in "_", are also available for value-only output (i.e. ZSTR_, LSTR_, UDEC_).

Symbols you can define to modify DEBUG behavior

CON Symbol	Default	Purpose
DOWNLOAD_BAUD	2_000_000	Sets the download baud rate.
DEBUG_COGS	%11111111	Selects which cogs have debug interrupts enabled. Bits 7..0 enable debugging interrupts in cogs 7..0.
DEBUG_DELAY	0	Sets a delay in milliseconds before your application runs and begins transmitting DEBUG messages.
DEBUG_PIN	62	Sets the DEBUG serial output pin. For DEBUG windows to open, DEBUG_PIN must be 62.
DEBUG_BAUD	DOWNLOAD_BAUD	Sets the DEBUG baud rate. May be necessary to add DEBUG_DELAY if DEBUG_BAUD is less than DOWNLOAD_BAUD.
DEBUG_TIMESTAMP	undefined	By declaring this symbol, each DEBUG message will be time-stamped with the 64-bit CT value.
DEBUG_LOG_SIZE	0	Sets the maximum size of the 'DEBUG.log' file which will collect DEBUG messages. A value of 0 will inhibit log file generation.
DEBUG_LEFT	(dynamic)	Sets the left screen coordinate where the DEBUG message window will appear.
DEBUG_TOP	(dynamic)	Sets the top screen coordinate where the DEBUG message window will appear.
DEBUG_WIDTH	(dynamic)	Sets the width of the DEBUG message window.
DEBUG_HEIGHT	(dynamic)	Sets the height of the DEBUG message window.
DEBUG_DISPLAY_LEFT	0	Sets the overall left screen offset where any DEBUG displays will appear (adds to 'POS' x coordinate in each DEBUG display).
DEBUG_DISPLAY_TOP	0	Sets the overall top screen offset where any DEBUG displays will appear (adds to 'POS' y coordinate in each DEBUG display).
DEBUG_WINDOWS_OFF	0	Disables any DEBUG windows from opening after downloading, if set to a non-zero value.

Simple DEBUG example in Spin2

```
CON _clkfreq = 10_000_000      'set 10 MHz clock (assumes 20 MHz crystal)

PUB go() | i
  REPEAT i FROM 0 TO 9         'count from 0 to 9
    DEBUG(UDEC(i))             'debug, output i
```

When run with Ctrl-F10, the Debug window opens and this is what appears:

```
Cog0  INIT $0000_0000 $0000_0000 load
Cog0  INIT $0000_0D6C $0000_10BC jump
Cog0  i = 0
Cog0  i = 1
Cog0  i = 2
Cog0  i = 3
Cog0  i = 4
Cog0  i = 5
Cog0  i = 6
Cog0  i = 7
Cog0  i = 8
Cog0  i = 9
```

In the first line of the report, you see Cog0 loading the Spin2 set-up code from $00000. In the second line, the Spin2 interpreter is launched from $00D58 with its stack space starting at $0101C. After that, the Spin2 program is running and you see 'i' iterating from 0 to 9.

If you change the "9" to "99" in the REPEAT, data will scroll too fast to read, but by adding a DLY command at the end of the DEBUG statement, you can slow down the output:

```
    debug(udec(i), dly(250))     'debug, output i with a 250ms delay after each report
```

Let's say you want to limit the messages being output, so that only odd values of 'i' are shown. You could use an IF at the start of your DEBUG statement to check the least-significant bit of 'i'. When the IF is false, no message will be output, causing only the odd values of i to be shown:

```
    debug(if(i & 1), udec(i), dly(250))     'debug, output only odd i values with a 250ms delay after each report
```

Simple DEBUG example in PASM

```
CON _clkfreq = 10_000_000      'set 10 MHz clock (assumes 20 MHz crystal)
```